

Intro2SE - Project Proposal - 6

Project Name

The student team is required to complete the **Project Proposal** documentation for the assigned course project, following the attached template.



Software Engineering Department
Faculty of Information and Technology
VNU-HCMUS

Table of Contents

Objectives	1
1 Member Contribution Assessment	2
2 Preliminary Problem Statement	3
3 Proposed Solution	4
4 Development Plan	5
5 Human Resources & Costing Plan	7
6 Tools setup	8
7 AI Usage Declaration	10
8 Presentation	11
9 Reflective Report	12

Project Proposal

Objectives

This document focus on the following topics:

- ✓ Completing the Project Proposal document with the following sections:
 - Preliminary Problem Statement
 - Proposed Solution
 - Development Plan
 - Human Resources & Costing Plan
- ✓ Understanding the Project Proposal document.
- ✓ This document will be used as input for AI tools to verify the quality of subsequent project artifacts.

1 Member Contribution Assessment

24127252 - Nguyễn Khánh Toàn - Contribution (25%)

- Task Details: Authored Section 2: Preliminary Problem Statement, configured targeted Operating Environment parameters, and engineered functional User Need / Autonomous Agent Skill Maps

-Visual Evidence:

✓ AP-8	Preliminary System Scoping & Problem Conte...	Toàn Nguyễn Khánh	Đinh Trí	Medium	DONE ✓	Done
AP-18	Design and draft the functional User Need ...	Toàn Nguyễn Khánh	Đinh Trí	Medium	DONE ✓	Done
AP-17	Authored the Section 2: Preliminary Proble...	Toàn Nguyễn Khánh	Đinh Trí	Medium	DONE ✓	Done
AP-16	Formulate and define the target Operating E...	Toàn Nguyễn Khánh	Đinh Trí	Medium	DONE ✓	Done

My workspace / ATS PROJECT - Orchestrated LLM Agent - Functional Specification

Nguyen Khanh Toan
 Last edited by [Nguyen Khanh Toan](#) on Jun 13, 2026

Orchestrated LLM Agent — Functional Specification

AI Recruitment Assistant / ATS Analytics System
Version 1.0

1. Overview

The **AI Orchestration Engine** is the core intelligence layer of the ATS Analytics system. It is implemented as a **multi-agent orchestrator** built on top of an LLM orchestration framework (LangChain or LlamaIndex), responsible for transforming raw, unstructured candidate data (PDF CV, LinkedIn profile, GitHub profile) into a structured, scored, and ranked candidate profile.

1.1 Orchestration Model

- **Pattern:** Orchestrator–Worker (a central Orchestrator Agent coordinates several specialized Sub-Agents/Tools).
- **Execution model:** Hybrid — some sub-agents run **sequentially** (output of one is input to the next), while independent enrichment tasks (LinkedIn, GitHub) run **in parallel**.
- **Trigger:** Activated by the `cv.uploaded` event published to the Message Queue (RabbitMQ/Kafka) after a CV is ingested via the Profile Ingestion Service.

- Link [Orchestrated LLM Agent - Functional Specification](#)

24127254 - Hồ Đình Trí - Contribution (25%)

- Visual Evidence:

HCMUS | SE Dept.

24127337 - Nguyễn Tiến Cường – Contribution (25%)

- Task Details: Formulated the forward-looking software development plan phases encompassing the deep Requirements Analysis, Software Design, and system Core Implementation lifecycles.

- Visual Evidence:

✓ AP-7 Development Lifecycle Planning for Core Engi...	CT Cường Nguyễn Tiến	Đinh Trí	Medium	DONE ✓	Done	Jun 13, 2026, 1:27 PM	Jun 13, 2026, 1:50 PM
AP-22 Outline and author the Implementation pha...	CT Cường Nguyễn Tiến	Đinh Trí	Medium	DONE ✓	Done	Jun 13, 2026, 1:40 PM	Jun 13, 2026, 1:53 PM
AP-21 Outline and author the Software Design pha...	CT Cường Nguyễn Tiến	Đinh Trí	Medium	DONE ✓	Done	Jun 13, 2026, 1:40 PM	Jun 13, 2026, 1:53 PM
AP-20 Outline and author the Requirements Analy...	CT Cường Nguyễn Tiến	Đinh Trí	Medium	DONE ✓	Done	Jun 13, 2026, 1:40 PM	Jun 13, 2026, 1:53 PM

24127382 - Trần Nhật Hoàng – Contribution (25%)

- Task Details: Designed Section 3: Proposed Solution, mapped the hybrid LLM Orchestrator execution loop architecture with MCP Host layers, and validated hardware runtime equipment standards.

- Visual Evidence:

✓ AP-9 Comprehensive Architectural Engineering & Pr...	HT Hoàng Tran	Đinh Trí	Medium	DONE ✓	Done
AP-10 Formulate the Hardware Specification Layer...	HT Hoàng Tran	Đinh Trí	Medium	DONE ✓	Done
AP-15 Authored the core conceptual definition for ...	HT Hoàng Tran	Đinh Trí	Medium	DONE ✓	Done
AP-14 Detail the Software Architecture Layer, defi...	HT Hoàng Tran	Đinh Trí	Medium	DONE ✓	Done

2

Preliminary Problem Statement

Written by: 24127252 - Nguyễn Khánh Toàn

Edited by: 24127337 - Nguyễn Tiến Cường

Reviewed by: 24127382 - Trần Nhật Hoàng

2.1. Business & Core Problem Description

In modern enterprise recruitment environments, processing pipelines depend heavily on velocity, parsing precision, and unified evaluator feedback to lock down high-caliber engineering professionals. As technical profiles span increasingly dense matrices of cross-platform documentation, ranging from raw text PDF CVs to active programming portfolios (GitHub) and external professional networking records (LinkedIn), human resource groups face severe text-processing bottlenecks.

Traditional Applicant Tracking Systems (ATS) operate primarily as passive, data-entry CRUD applications ("Old School" architectures). Even current commercial tools marketed as "Smart Systems" fall short under enterprise loads; they rely on fragile, keyword-matching heuristic rules or superficial LLM prompt wrappers that completely lack contextual awareness, structural semantic flexibility, and automated tool-execution competence.

Consequently, modern enterprise environments face distinct technical friction points:

- **Context-Blind Resume Appraisals:** Legacy keyword parsers completely strip away engineering context, failing to properly evaluate open-source codebase contributions, repository architectural layouts, or unstructured project documentation (such as GitHub README.md quality markers).
- **High Administrative Overhead & Data Fragmentation:** Recruiters must manually harvest metadata across disconnected external platforms, resulting in data silo fragmentation, manual workflow delays, and prolonged corporate Time-to-Hire metrics.
- **Disconnected Evaluation Flows:** Hiring panels lack real-time synchronization or visual tools (such as dual-screen split workspaces) to directly verify AI-generated summaries against raw candidate text. Furthermore, without automated threshold alerts, conflicting review scorecards pass completely unnoticed, degrading interview consensus consistency.
- **Data Privacy Vulnerabilities:** Standard setups expose sensitive candidate parameters (e.g., historical compensation records or personal contact data) to baseline technical evaluators, posing a data handling risk.

2.2. Operating Environment

To guarantee performance, multi-user concurrency, and secure processing pipelines, the system is engineered to run reliably within the following environments:

- **Client Interface:** Modern responsive web browsers fully compliant with the HTML5 markup engine and the ECMAScript 2024 runtime execution standard.
 - **Host Environment:** Linux-based cloud infrastructure deployed on Microsoft Azure (Azure App Service, Azure Blob Storage), coordinating an asynchronous Modular Monolithic + Clean Architecture fabric engineered using Node.js (NestJS) for the API gateway layer and Python (FastAPI) for AI and data processing modules. The multi-agent pipeline adopts an Event-Driven coordination model managed by a central Orchestrator to ensure fault-tolerance and load scalability.
- **Agent Runtime Environment:** Claude 3.5 Sonnet / GPT-4o native runtime execution structures, utilizing Model Context Protocol (Node-MCP) host configurations to discover and execute runtime infrastructure tools dynamically.

2.3. Design & Implementation Constraints

Development operations and deployment architectures must strictly adhere to the following technological boundaries:

- **Core Programming Languages:** Type-safe TypeScript for web applications and API gateway patterns, combined with Python for data manipulation and AI workflows.
- **Architecture Pattern:** Modular Monolithic + Clean Architecture, with the project segmented into independently deployable modules (CV Parser, GitHub Analyzer, LinkedIn Analyzer, AI Analytics, Notification). The multi-agent processing layer follows an Event-Driven model: when a CV is uploaded, the Orchestrator emits a queued event carrying the candidate UUID; downstream service modules consume the event independently. If any module encounters a failure, the message is retained in the queue and re-processed upon recovery, ensuring resilience under high request volumes.
- **Model Context Protocol (MCP) Standard:** All AI orchestrations, agentic tool configurations, and system-to-model communications must adhere to MCP Protocol Version 2024-11-05.
- **Database & Persistence Layers:** PostgreSQL relational database engines to manage structured transactional data, extended by the pgvector extension to store semantic context embeddings and compute high-dimensional proximity distances (Smatch).
- **Cloud Infrastructure:** Microsoft Azure is the designated cloud provider (replacing AWS) to leverage pre-existing team account access and eliminate payment configuration overhead. Core services include Azure Blob Storage for CV file management, RabbitMQ/Kafka as the message queue backbone, and Azure App Service for application hosting.
- **Version Control & Repository Layout Standards:** All development output must be structured within a team repository using standardized paths: source directories under /src and versioned documentation files under /docs.

2.4. Design & Implementation Constraints

To effectively address the data bottlenecks, evaluation silos, and privacy vulnerabilities identified in Section 2.1, the functional scope of the ATS Talent Analytics System is structured into an incremental delivery roadmap (Figure 2.4).

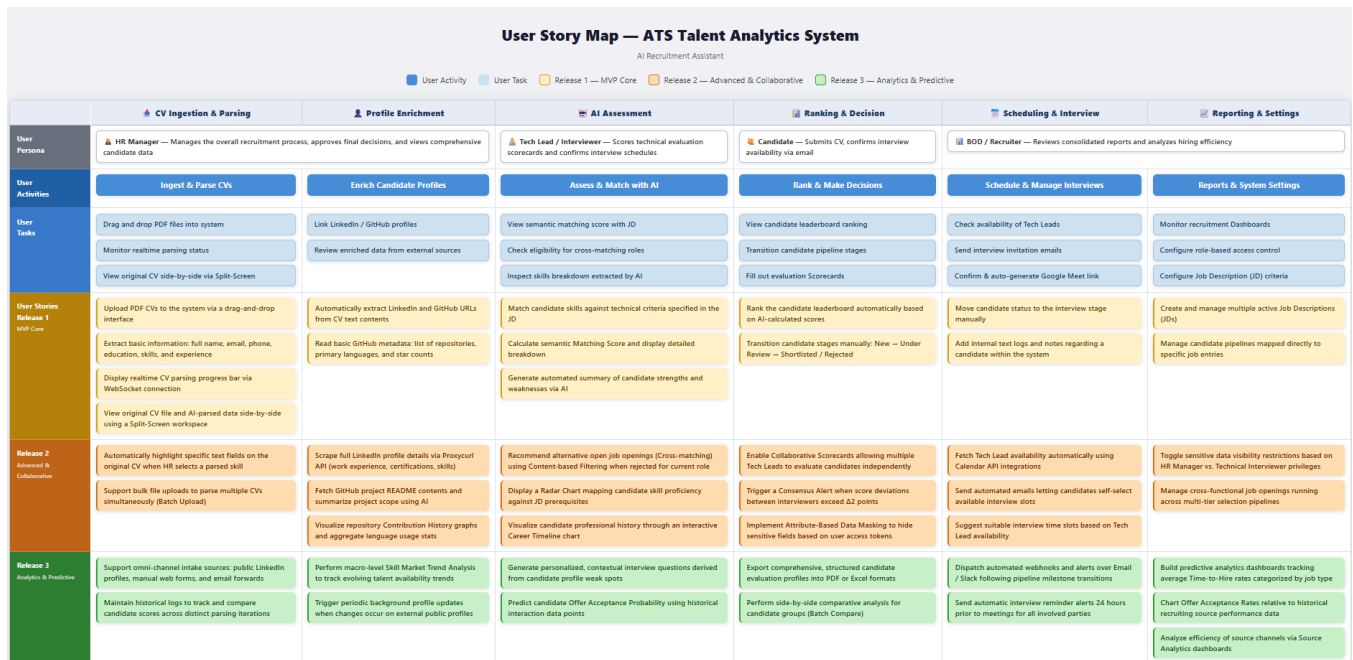


Figure 2.4: Functional Roadmap and Feature Mapping of the ATS Talent Analytics System.

The implementation is executed across three strategic phases:

- **Release 1 - MVP Core:** Targets administrative overhead and evaluation disconnects. Delivers core PDF parsing, basic GitHub metadata extraction, semantic match-scoring (Smatch), and the dual-screen split workspace.
- **Release 2 - Advanced & Collaborative:** Resolves data privacy risks and scheduling friction. Introduces deep LinkedIn profile scraping, Attribute-Based Data Masking, and automated Tech Lead availability checking via Calendar APIs.
- **Release 3 - Analytics & Predictive:** Deploys the advanced intellectual layer. Features predictive analytics (Offer Acceptance Probability), macro-level skill market trend analysis, and contextual interview question generation.

3

Proposed Solution

3.1 Software

3.1.1. Features

Written by: 24127382 - Trần Nhật Hoàng
Edited by: 24127254 - Hồ Đình Trí
Reviewed by: 24127252 - Nguyễn Khánh Toàn

SmartATS has hybrid architecture with LLM taking the role as the orchestrator. Deterministic pipeline handles high-volume data processing tasks.

Traditional pipelines are responsible for resume parsing, vector indexing, and candidate data persistence for reliability. On top of that, an AI agent will select tools, retrieve relevant information through MCP, help coordinate workflows such as candidate screening, semantic ranking, interview scheduling, and conversational assistance.

This design combines the efficiency of deterministic systems with the flexibility of agent-based reasoning, enabling intelligent automation without sacrificing performance or operational cost.

User Need (Nhu cầu)	Autonomous Agent Skill & Tool Requirement (Yêu cầu)
As an HR Manager, I want to paste a job description and automatically fetch/rank candidate CVs from Google Drive without manual configuration.	Skill: Autonomous Sourcing & Semantic Retrieval Tool: `drive_search_mcp` to autonomously crawl folders, parse PDFs into semantic vectors, and generate context rankings.
As an HR Manager, I want to find suitable candidates to the jobs without reading through hundreds of CV	Skill: Contextual Semantic Filtering & Ranking Tool: `rag` to find candidates based on semantics relevance

As a HR Manager, I want to talk to an interactive bot that knows my progress and can autonomously book an interview window on the team's calendar.	Skill: Contextual Scheduling Orchestration Tool: `google_calendar_api` and `slack_notify_mcp` executed autonomously by the main program when candidate slots align.
--	---

3.1.2. Software Architecture

Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127254 - Hồ Đình Trí

Reviewed by: 24127252 - Nguyễn Khánh Toàn

Orchestrator Loop operating through four core components:

1. **Core Agent Runtime:** maintain execution loop, session memory.
2. **Model Context Protocol (MCP) Host Layer:** providing interface for the agent to interact with external systems including file repositories, vector databases, communication services, and scheduling platforms .
3. **Custom Tool & Skills Registry:**
 - `ResumeParserSkill(pdf\_bytes)`: Extracts schema-validated candidate representations.
 - `CandidateRankingSkill(job\_description)`: Retrieves and ranks based on semantic relevance.
 - `CalendarOrchestrationSkill(interviewer\_ids, date\_range)`: Solves scheduling availability dynamically.
0. **Enterprise State Machine:** A deterministic state layer validating the agent's proposed executions before persisting transactions to PostgreSQL.

3.2 Hardware

Written by: 24127382 - Trần Nhật Hoàng

Edited by: 24127254 - Hồ Đình Trí

Reviewed by: 24127252 - Nguyễn Khánh Toàn

- Infrastructure: Cloud-allocated VPS (Minimum 2 vCPU, 4GB RAM) running the ATS services and agent runtime .
- MCP Layer: FastMCP servers for resume retrieval, candidate searching, scheduling.
- LLM API Infrastructure: Open-source models through Hugging Face API for resume summary
- Data Layer: PostgreSQL for structure data and pgvector for semantics retrieval
- File Storage: Local or S3-compatible storage for resumes uploading

4 Development Plan

In modern enterprise recruitment systems, automating candidate evaluation faces a critical bottleneck: Large Language Models (LLMs) operating in an "End-to-End" fashion (inputting all data into a single prompt) often suffer from context loss, low accuracy, and a "black-box" effect with limited interpretability. Furthermore, data fragmentation across multiple platforms severely degrades system stability.

To resolve this, SmartATS proposes a **Multi-Agent LLM Framework**. Instead of monolithic evaluation, this architecture decouples the processing pipeline into a network of independent Sub-Agents. Each agent is responsible for extracting a specific competency criterion and standardizing it into a Structured Representation before the system computes the final score. This approach forces the AI's reasoning path to strictly align with human HR verification processes, ensuring absolute transparency, reliability, and comprehensive auditability.

4.1 Requirements Analysis

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127254 - Hồ Đình Trí

This document defines the core specifications for the SmartATS system. Requirements are categorized by priority: **P0** (Critical/Must-have) and **P1** (Value-add/Should-have).

4.1.1 Functional Requirements

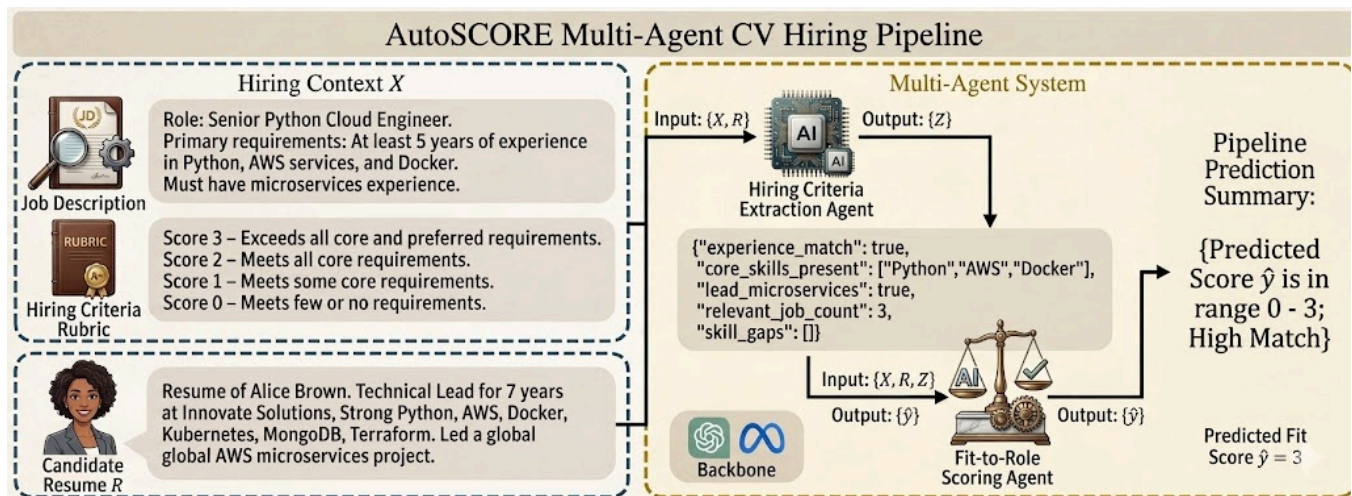
- **[P0] Ingestion & Parallel Enrichment:** The system must process standard input streams by parsing uploaded PDF CVs while simultaneously triggering asynchronous crawlers to extract metadata from the candidate's LinkedIn profile and GitHub repository.
- **[P0] Multi-Agent Component Extraction:** The system must initialize independent Sub-Agents corresponding to each criterion in the Evaluation Rubric. Each Agent scans the profile to extract specific evidence and standardizes it into a structured format (JSON schema - variable **Z**).
- **[P0] Cross-Verification Scoring & Justification:** The core Scoring Agent must simultaneously receive the JSON payload (**Z**), the Evaluation Rubric (**X**), and the Original Raw Document (**R**). Since the JSON extraction (**Z**) by Sub-Agents may encounter errors or AI hallucinations, the system must utilize the raw document (**R**) as the ground-truth. In case of anomalies, the Agent must fallback to **R** to auto-correct deviations before finalizing the score and generating justification logs.
- **[P1] Interactive Analytics:** The admin dashboard must provide dynamic charts (SVG/Canvas) to visually evaluate programming language ratios and skill metrics.
- **[P1] Automated Scheduling:** An interactive bot integrating the Google Calendar API must scan the Tech Leads' availability and automatically propose suitable interview slots.

4.1.2 Non-Functional Requirements

- **Traceability & Auditability:** The data processing flow must be fully traceable. Any generated score must be traceable back to the original text snippet in the CV via an Object ID.
- **Scalability & Throughput:** The system must implement a Message Broker (RabbitMQ/Kafka) to transition from synchronous to asynchronous processing, ensuring the API Gateway does not timeout during bulk CV uploads.
- **Fault Tolerance (SLA):** The system must enforce an auto-retry policy (up to 3 times) for failed API/Tool calls before routing the task to a Dead Letter Queue for manual review.
- **Data Security & Privacy:** The system must enforce Attribute-Based Access Control (ABAC) to automatically mask (***) Personally Identifiable Information (PII) on the technical interviewers' dashboard.

4.2 Software Design

To handle high-volume processing and LLM flexibility, SmartATS adopts a **Modular Monolith + Clean Architecture** deployed on Microsoft Azure. The data flow strictly adheres to the **Multi-Agent Processing Pipeline**:



Note: This image was generated by the my team using Gemini's "banana" tool; please note that "banana" only supports drawing/illustration, not conceptual ideas.

Standard Operating Pipeline:

- **Step 1 & 2: Trigger & Parallel Enrichment:** The uploaded file passes through the API Gateway (NestJS), is pushed to Azure Blob Storage, and triggers an event via RabbitMQ. Worker Nodes (PDF parser, GitHub scraper, LinkedIn crawler) run in parallel to synthesize the Task Context (X & R).
- **Step 3: Multi-Agent Component Extraction:** The AI Orchestrator (Python/FastAPI) spawns multiple Sub-Agents. Each Agent scans the Context to find evidence for a single, specific criterion. This evidence is then encoded into a static data structure (JSON Representation - Z).
- **Step 4: Scoring & Error Correction:** The central Scoring Agent receives three inputs: the Rubric (X), the JSON structure (Z), and the Raw Data (R). The *R-Fallback Verify* mechanism is triggered here: the Agent uses Z as a framework to accelerate processing but constantly cross-references the original document (R) to self-correct any extraction errors from Step 3. This ensures absolute accuracy for the final score (S_{match}) and Justification logs.
- **Step 5: Persistence & Analytics:** Evaluated data is persisted in PostgreSQL (using `pgvector` for embeddings). Background metrics push signals to the Frontend to render SVG/Canvas charts.

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127254 - Hồ Đình Trí

4.3 Implementation

Written by: 24127337 - Nguyễn Tiến Cường

Edited by: 24127382 - Trần Nhật Hoàng

Reviewed by: 24127254 - Hồ Đình Trí

The execution plan follows an Agile methodology, divided into three sequential Releases (CI/CD) to mitigate integration risks and deliver continuous value.

Phase 1: MVP Core - Core Processing Foundation

Objective: Finalize the fundamental document processing flow and ensure the absolute accuracy of the Multi-Agent model via cross-verification.

- **Infrastructure & Database:** Provision cloud environments on Microsoft Azure (App Service, Blob Storage). Initialize relational schemas on PostgreSQL and configure the `pgvector` extension for semantic embeddings.
- **Core Backend & Multi-Agent Architecture:** Build the Node.js (NestJS) API Gateway. At the AI core (Python/FastAPI), deploy the initial **Extraction Sub-Agents** to parse PDFs into JSON (Z). Implement the **Scoring Agent** featuring the *R-Fallback Verification* algorithm to eliminate AI hallucinations.
- **UI/UX:** Develop the Dual-screen workspace, allowing HR Managers to view the original PDF alongside the extracted JSON/transparent scores simultaneously.

Phase 2: Advanced & Collaborative - Data Ecosystem & Security

Objective: Transition to an asynchronous architecture, expand automated data pipelines, and establish enterprise-grade security barriers.

- **Async Event-Driven Architecture:** Deploy RabbitMQ as the system's backbone, converting all CV uploads into queued events to prevent API timeouts during high-traffic loads.
- **Cross-platform Enrichment:** Launch dedicated Worker Nodes consuming RabbitMQ messages to automatically scrape LinkedIn profiles and analyze GitHub repository quality (including README .md evaluations).
- **Security & ABAC:** Implement Attribute-Based Data Masking. This algorithm automatically detects and masks PII, emails, phone numbers, and expected salary fields when accessed by a "Technical Interviewer" role.
- **Automated Scheduling:** Integrate Google Calendar API and Slack Webhooks to automatically scan Tech Leads' available slots and propose interview schedules.

Phase 3: Analytics & Predictive - Advanced Intelligence Layer

Objective: Elevate the system from "data processing" to "strategic decision support" through visualization and predictive AI.

- **Interactive Analytics Dashboard:** Design and embed real-time SVG/Canvas interactive charts into the HR dashboard to visualize programming language distributions, skill match ratios, and macro recruitment trends.
- **Predictive Modeling:** Train and deploy the Offer Acceptance Probability algorithm based on historical data and cultural/skill alignment metrics.
- **Contextual Q&A Generation:** Maximize the utility of the JSON structural data from Phase 1. The AI will automatically identify competency "gaps" or weak evidence in the candidate's CV to generate a highly targeted list of technical interview questions, significantly reducing the interview panel's preparation time.

4.4 Testing

Written by: 24127254 - Hồ Đình Trí

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127337 - Nguyễn Tiến Cường

4.4.1 Testing Strategy

To ensure the reliability of the core analytical features and async pipelines, the system utilizes a multi-layered testing strategy:

-Unit Testing: Focuses on the inner layers of the Clean Architecture (**Domain** and **Application** layers). It validates pure business logic, such as the *Job Matching Score algorithm*, semantic profile evaluation, and regex link parsers, completely isolated from external frameworks or database connections.

-Integration Testing: Verifies the interoperability between decoupled modules and the infrastructure layer. This ensures that the in-memory event-driven system successfully triggers the *Data Enrichment Module* (GitHub/LinkedIn parsing) once an ingestion event occurs.

-System Testing (End-to-End): Simulates real-world HR workflows. The test path starts from uploading a raw PDF resume, verifying multi-channel data harvesting, and ensuring the final structural analytical results are pushed to the UI via WebSockets in real time.

4.4.2 Core Test Cases Specification

ID	Component	Description	Input Data	Expected Outcome	Status
1	Ingestion & Security	Validate system constraints on raw resume file uploads.	A non-PDF file or a PDF exceeding 10MB.	The API Gateway rejects the request with a HTTP 400 bad request, showing a clear error message. No processing jobs are queued.	pass
2	Parsing & Event-Driven	Verify asynchronous execution of the AI parsing workflow.	A valid resume file	Immediately returns a 202 Accepted status to the user interface. Background processing initiates and successfully structures raw text into a standard JSON schema.	pass
3	Data Enrichment Engine	Test cross-channel profile enrichment via external public APIs.	A valid public GitHub URL extracted from the candidate's CV.	The system calls the GitHub REST API, extracts the repository's README.md, runs the NLP engine for summarization, and dynamically builds the programming language ratio chart.	pass
4	Field-Level	Verify dynamic	An	The workspace view hides	pass

	Security	Field-Level Attribute-Based Access Control (ABAC).	authentication on token possessing the Technical Interviewer role.	confidential identifiers (Email, Phone, Expected Salary) using data masking (***), while showing all technical skills charts.	
--	----------	--	--	---	--

4.5 Deployment and Maintenance

Written by: 24127254 - Hồ Đình Trí

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127337 - Nguyễn Tiến Cường

4.5.1 Deployment Architecture

The system adopts a **Modular Monolith** deployment strategy to minimize operational overhead while guaranteeing seamless service delivery within the project schedule.

-Containerization: The entire software ecosystem (Frontend, Core Analytics Backend, and PostgreSQL Database) is encapsulated using **Docker containers**. This eliminates environment drift and ensures identical runtime execution between local development and production environments.

-Target Infrastructure: Orchestrated via docker-compose on a secure virtual instance. The persistent analytical records are securely isolated in high-performance local volumes.

4.5.2 Maintenance and Monitoring Policies

Since the core value of the system relies on AI accuracy and uninterrupted background jobs, the maintenance pipeline is split into two operational frameworks:

-System Performance Monitoring & Failure Fallback: * **Centralized Logging:** Captures full stack traces of API transactions, WebSocket synchronization, and third-party interactions (GitHub/LinkedIn requests).

+**Retry Mechanisms:** In case of failure during the asynchronous AI processing phase, the event handler schedules up to 3 automatic retries. If the error persists, the state machine transitions to FAILED, generating a fallback alert that prompts the HR manager to initiate a manual resume review.

-AI Model Drift & Dictionary Maintenance: * Technical and market terminologies evolve over time, introducing "model drift" where new skills may not be accurately mapped by stagnant prompts.

+Every quarter, the engineering team reviews the **Audit Trail** records (where HR users manually modified structural errors made by the AI). This data serves as the ground-truth telemetry to calibrate the system's LLM prompt templates and enrich the technical phrase dictionary without altering the underlying domain code.

5 Human Resources & Costing Plan

Written by: 24127254 - Hồ Đình Trí

Edited by: 24127337 - Nguyễn Tiến Cường

Reviewed by: 24127382 - Trần Nhật Hoàng

To fully align with the accelerated lifecycle, the team operates as a **cross-functional engineering unit** where every member actively contributes across Frontend, Backend, and Integration layers to eliminate single points of failure:

- **24127254 - Hồ Đình Trí (Full-Stack & QA/DevOps Lead):** Coordinates core workspace containerization, implements ABAC security filtering mechanisms, manages event-driven notifications, and structures automated system validation suites.
- **24127252 - Nguyễn Khánh Toàn (Full-Stack & Integration Engineer):** Engineers the interactive split-screen layout interfaces, handles data synchronization with external provider APIs, and deploys the automated interviewer calendar allocation systems.
- **24127337 - Nguyễn Tiến Cường (Full-Stack & Data Analytics Engineer):** Deploys interactive SVG/Canvas analytical data charts, constructs core business relational entity schemas, and manages background system performance metrics.
- **24127382 - Trần Nhật Hoàng (Full-Stack & AI Orchestration Engineer):** Develops the core LLM execution loop controllers, designs custom MCP skill servers for resume processing, and optimizes vector indexing queries using pgvector.

Anticipated Project Costs:

- **LLM Inference & Token Budget:** \$25.00 allocated for testing autonomous tool invocations, structural JSON generation, and prompt pipeline evaluations.

- **Cloud Infrastructure:** \$0.00 (Utilizing free-tier compute credits on Render, Vercel, or local self-hosted environments for continuous component staging).

6

Tools setup

Written by: 24127254 - Hồ Đình Trí

Edited by: 24127252 - Nguyễn Khánh Toàn

Reviewed by: 24127337 - Nguyễn Tiến Cường

Professional software engineering tools are integrated and automated to audit team performance, verify operational health, and secure systematic artifact tracking:

- **Moodle:** Official academic gateway for managed milestone evaluations.
- **JIRA:** Configured with an Agile Scrum framework to map vertical feature slices across explicit milestone sprints (Sprint 1 through Sprint 3).
- **GitHub Verified Directory Structure:**
 - **/src** - Houses the production-grade source code including frontend workspace dashboards, core analytics backend servers, and custom MCP tool descriptors.
 - **/docs/management** - Preserves JIRA sprint velocity graphs, retrospective items, and project verification images.
 - **/docs/requirements** - Stores core agile user stories, prompt definition specs, and functional agent capability blueprints.
 - **/docs/analysis and design** - Contains entity relationship models, architectural sequence diagrams, and clean architecture maps.
 - **/docs/test** - Logs automated script assertions, unit/integration verification reports, and system validation parameters

7

AI Usage Declaration

AI assets are fully leveraged within professional boundaries. LLM tools have been explicitly utilized for synthesizing structural mock JSON schemas, writing unit tests for custom MCP server integrations, and reviewing system prompt strategies to avoid context leakage. All primary system code, custom script mappings, and software architecture diagrams represent the group's authentic engineering designs.

8

Presentation

- Video Link: <https://youtu.be/LI1-cGTxMUo>

9

Reflective Report

The evaluation of this project proposal template reveals distinct components that are highly actionable for execution, alongside standard metrics that require modernization for agentic software:

-Most Helpful Sections: **Section 3 (Proposed Solution)** and **Section 6 (Tools Setup)** serve as the critical baseline for our architecture. Specifying the Model Context Protocol (MCP) host parameters and defining custom registries (ResumeParserSkill, CandidateRankingSkill) early in the timeline prevents technical drift. Furthermore, enforcing a strict directory layout on GitHub (/src, /docs/requirements, /docs/test) ensures all generated engineering artifacts align seamlessly with software engineering auditing standards.

-Obsolete Components: Traditional static resource costing structures are somewhat outdated for autonomous systems. Unlike deterministic platforms with fixed cloud hosting fees, an agentic loop behaves dynamically. The project cost is heavily dependent on variable LLM API token consumption model parameters driven by autonomous tool invocations. However, retaining this section helps maintain strict fiscal and architectural boundaries during rapid system prototyping.